

LAB4 실험 전 보고서

FPGA4 / MCU0 — 4비트 곱셈기 · BCD→이진수 변환 · STM32 LED와 버튼 (과제1·2)

이름
형종희학번
2023440145조
C조작성일
2026-10-07실험일
2026-10-12FPGA 보드
Combo II-DLD S75 ·
xc7s75fpga484-1MCU 보드
NUCLEO-G474RE
(STM32G474RE)도구
Vivado 2026.1 · Icarus
Verilog 12.0 · VS Code
+ STM32Cube 확장
3.11.0

GitHub 제출 정보

항목	값
저장소	https://github.com/hyeongjonghee/electronics2-LABs (Public)
LAB4 설계 소스 커밋	f578e2d — FPGA 2개 회로의 RTL · 테스트벤치 · XDC · 빌드 결과, STM32 과제1·2 코드 (24개 파일)
LAB4 경로	LAB4/circuits/LAB4-01~02/ , LAB4/stm32/task1~2/ , LAB4/evidence/ , LAB4/LAB4_실험전보고서.html · .pdf
실험 전 보고서 태그	lab4-prelab — 이 보고서(HTML · PDF)가 들어간 커밋을 가리킴. 최종 제출 태그 <code>lab4-submit</code> 은 실험 후 보고서 커밋에 따로 건다.
STM32 기반 프로젝트	Glaysia/ece2_474 태그 <code>v1.1.0</code> = 커밋 <code>dd94f01</code> (수업 지정 버전). 학생이 바꾼 파일은 <code>Core/Src/ece2_474_routine.c</code> 하나이며, 과제별 사본을 위 저장소에 보관.

0. 사전 결과 요약

LAB4는 10/12 수업(FPGA4 / MCU0)의 실습이다. 수업 안내에 따라 **FPGA 교재 실습 2개**(곱셈기, BCD 변환)와 **STM32_0 직접 작성 과제 2개**(과제1, 과제2)를 준비했다. FPGA 회로는 두 시뮬레이터(Vivado XSim, Icarus)의 자기검사를 통과했고 비트스트림까지 만들었다. STM32 과제는 실제 프로젝트로 빌드했고, 보드 없이 동작을 확인하기 위해 같은 C 함수를 PC에서 가짜 시계·가짜 버튼과 함께 실행하는 모의 실행으로 타이밍을 검증했다.

#	실험	검증	결과	자원 / 빌드
1	4비트 곱셈기 (FPGA)	XSim · Icarus 자기검사	PASS 256개 입력 조합	LUT 16 · FF 0 · IOB 16 · DSP 0
2	BCD→이진수 변환 (FPGA)	XSim · Icarus 자기검사	PASS 유효 100 + 무효 156	LUT 7 · FF 0 · IOB 16 · DSP 0
3	STM32_0 과제1 · 버튼으로 간격 변경	빌드 + 호스트 모의 실행	PASS 1→2→4→8→1 s	FLASH 25,824 B (4.93%) · RAM 2,656 B
4	STM32_0 과제2 · 외부 LED (PB0)	빌드 + 호스트 모의 실행	PASS 과제1과 같은 타이밍	FLASH 25,824 B (4.93%) · RAM 2,656 B

공통 사항

- FPGA 두 회로는 클록이 없는 조합회로다.** 입력 스위치가 바뀌면 출력 LED가 곧바로 따라 바뀐다. 기억하는 칸(플립플롭)이 0개이고, 클록 제약이 없으므로 Vivado가 보고한 WNS · WHS 0.000은 "검사할 클록 경로 없음"을 뜻한다.
- FPGA 입출력:** 두 회로 모두 DIP SW1~8을 입력, LED1~8을 출력으로 쓴다. LAB2 · LAB3에서 보드로 검증한 핀(LVCMOS33)이라 새 핀 확인은 필요 없다.
- 설계 소스 출처:** FPGA 설계 모듈과 테스트벤치는 수업 저장소의 `docs/FPGA_additional.md` (v1.1.0)를 그대로 사용했다. 내가 추가한 것은 보드 연결용 최상위 모듈, XDC, 테스트벤치의 파형 저장 한 줄이다.

시연 전 확인할 것 — ① 교재 예제(`mult_4bit.zip` , `bcd_conv.zip`)의 장치 연결 방식과 이 보고서의 스위치/LED 배치가 같은지 대조한다. 다르면 XDC만 바꾼다.
② NUCLEO 보드에서 PB0이 나오는 헤더 위치를 보드 회로도로 확인한 뒤 외부 LED를 꽂는다.

목차

- | | |
|------------------------|-----------------------|
| 1. 실험 1 · 4비트 곱셈기 | 4. 실험 4 · STM32_0 과제2 |
| 2. 실험 2 · BCD → 이진수 변환 | 5. 참고문헌 |
| 3. 실험 3 · STM32_0 과제1 | |

1. 실험 1 · 4비트 곱셈기 (mult_4bit)

① 목적

4비트 수 두 개를 곱해 8비트 결과를 내는 조합회로를 만들고, 256개 입력 조합 전체에서 결과가 맞는지 확인한다. 보드에서는 스위치로 두 수를 넣고 LED 8개로 곱을 읽는다.

② 이론 · 설계 계산

결과가 왜 8비트면 충분한가. 4비트 부호 없는 수는 0~15이다. 가장 큰 곱은

$$P_{\max} = 15 \times 15 = 225 < 2^8 = 256$$

이므로 8비트(0~255)에 항상 들어간다. 일반적으로 n 비트 $\times$ n 비트 곱은 $2n$ 비트면 넘치지 않는다. $(2^n - 1)^2 < 2^{2n}$ 이기 때문이다.

하드웨어가 곱하는 방법. 초등학교 세로 곱셈과 같다. B 의 각 자리 비트 b_i 가 1이면 A 를 i 칸 왼쪽으로 민 값(부분곱)을 더한다 [1].

$$P = A \cdot B = \sum_{i=0}^3 b_i (A \cdot 2^i)$$

예를 들어 5×3 은 $B = 0011$ 이므로 $5 \cdot 2^0 + 5 \cdot 2^1 = 5 + 10 = 15$ 이다. 이진수로는 $0101 + 1010 = 0000\ 1111$ 이다.

RTL은 `assign product = a * b;` 한 줄이다. 곱셈 기호를 쓰면 합성 도구가 부분곱과 덧셈 회로를 대신 만든다. 이 회로는 크기가 작아서 Vivado가 DSP 블록 대신 LUT 16개로 구현했다(⑥ 참고).

③ 포트 · 보드 매핑

신호	최상위 포트	보드 장치	FPGA 핀
a[3:0] (MSB→LSB)	sw[7:4]	DIP SW1~4 (ON=1)	Y1 · W3 · U2 · T1
b[3:0]	sw[3:0]	DIP SW5~8	W4 · W1 · V4 · U4
m[7:0]	led[7:0]	LED1~8 (켜짐=1)	L4 · M4 · M2 · N7 · M7 · M3 · M1 · N5

IOSTANDARD는 모두 LVCMOS33 (LAB3 XDC와 같은 값). 클록 · 리셋은 쓰지 않는다.

④ RTL

```
`timescale 1ns/1ps
// 출처: Glaysia/ece2_474 v1.1.0 docs/FPGA_additional.md (1. 4비트 곱셈기)
// 4비트 곱셈: 입력 두 개를 곱해 8비트 결과를 만듭니다.
module multiply_unsigned #(parameter WIDTH = 4)(
    input wire [WIDTH-1:0] a, b,
    output wire [2*WIDTH-1:0] product
);
    assign product = a * b;
endmodule
```

```
`timescale 1ns/1ps
// 출처: Glaysia/ece2_474 v1.1.0 docs/FPGA_additional.md (1. 4비트 곱셈기)
module mult_4bit(input wire [3:0] a, b, output wire [7:0] m);
    multiply_unsigned #(WIDTH(4)) u_multiply(a, b, m);
endmodule
```

```
`timescale 1ns/1ps
// 보드 연결: a = SW1~4 (sw[7:4]), b = SW5~8 (sw[3:0]), m = LED1~8 (led[7:0])
module lab4_mult_4bit_top(input wire [7:0] sw, output wire [7:0] m);
    mult_4bit u_mult(.a(sw[7:4]), .b(sw[3:0]), .m(led));
endmodule
```

⑤ 시뮬레이션

테스트벤치는 a 를 0~15, 각 a 마다 b 를 0~15로 바꾸며 1 ns마다 한 쌍을 넣는다. 매번 m 이 $a \times b$ 와 다르면 즉시 오류로 멈춘다(\$fatal). 끝까지 가면 PASS를 출력한다.

시뮬레이터	출력	판정
Vivado XSim (build.tcl)	PASS multiplier: 256 input pairs	통과
Icarus Verilog 12.0	PASS multiplier: 256 input pairs · \$finish at 256 ns	통과
보고서 생성 시 VCD 재검사	Icarus wave.vcd 의 256개 구간에서 $m = a \times b$ 일치 256/256	통과

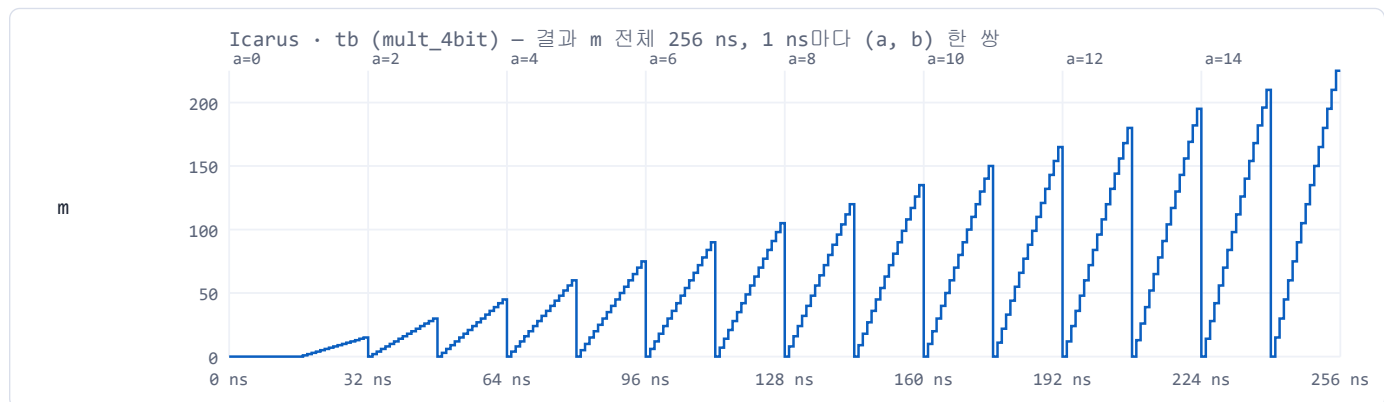


그림 1-1. 전체 파형. 16 ns 단위로 a 가 1씩 커지고, 그 안에서 b 가 0→15로 오르므로 m 은 기울기 a 인 계단이 된다. a 가 클수록 계단이 가파르고, 마지막 구간의 끝이 최댓값 225이다.

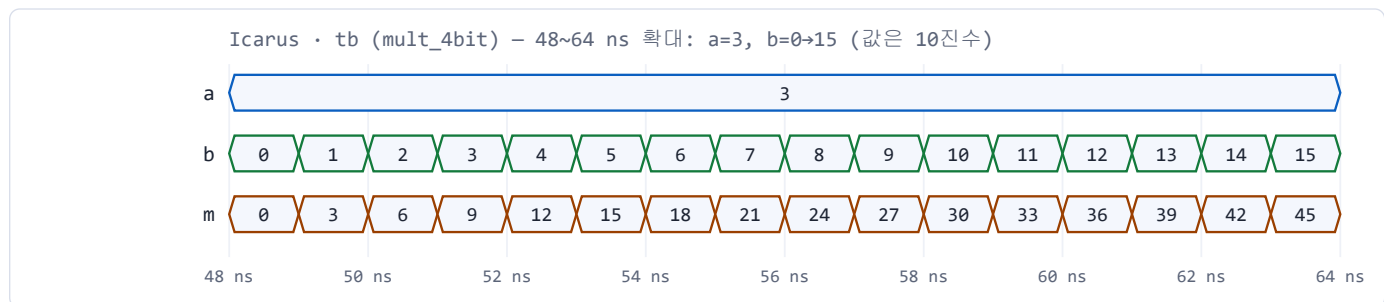


그림 1-2. $a = 3$ 구간. b 가 1 늘 때마다 m 이 3씩 늘어 0, 3, 6, ... 45가 된다.

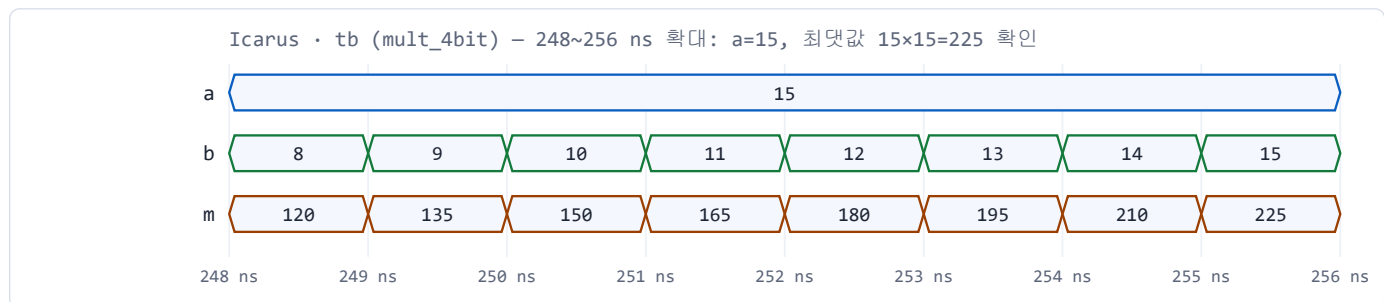


그림 1-3. 마지막 구간. $a = 15, b = 15$ 에서 $m = 225$ 로, 8비트 안에 들어가는 최댓값을 확인했다.

⑥ 합성 · 구현 결과 (사전 빌드)

항목	값
합성 · 구현	synth_design Complete · write_bitstream Complete · bit_exists=1
Slice LUT	16 / 48,000 (0.03%) — 13개는 O6 출력만, 3개는 O5·O6 두 출력 사용
Slice Register (FF)	0 — 조합회로
Bonded IOB	16 / 338 (입력 8 + 출력 8)
DSP · BRAM	0 · 0
타이밍	WNS 0.000 / WHS 0.000 — 클록 없음, 제약 대상 경로 없음

⑦ 보드 실습 계획

스위치를 위로 올린 것이 1이다. 아래 표의 스위치를 맞춘 뒤 LED가 같은지 본다. ●은 켜진 LED(왼쪽이 LED1 = m[7])이다.

a×b	SW1~4 SW5~8	기대 m	m (이진)	LED1~8
0×9	0000 1001	0	00000000	○○○○○○○○
1×9	0001 1001	9	00001001	○○○○●○○●
5×3	0101 0011	15	00001111	○○○○●●●●
12×10	1100 1010	120	01111000	○●●●●○○○
15×15	1111 1111	225	11100001	●●●○○○○●

0을 곱하면 전부 꺼지는지(0×9), 1을 곱하면 다른 쪽 수가 그대로 나오는지(1×9), 최댓값(15×15)에서 위 세 자리와 맨 아래 자리가 켜지는지를 차례로 확인한다.

2. 실험 2 · BCD → 이진수 변환 (bcd_conv)

① 목적

십의 자리와 일의 자리를 각각 4비트 BCD(0~9)로 받아 하나의 이진수(0~99)로 바꾼다. 각 자리가 0~9를 벗어나면 `valid` 를 0으로 내려 잘못된 입력임을 알린다.

② 이론 · 설계 계산

BCD란. 사람이 읽는 10진수의 각 자리를 4비트씩 따로 적는 방식이다. 예를 들어 42는 BCD로 `0100 0010` 이지만, 이진수로는 `010 1010` 이다. 두 표기는 같은 수를 다르게 적은 것이다. 계산기 화면(자리별 숫자)과 컴퓨터 내부(이진수) 사이를 잇는 다리가 이 회로다.

변환식. 자리값을 곱해 더하면 된다 [1].

$$N = 10T + O, \quad 0 \leq T, O \leq 9 \Rightarrow 0 \leq N \leq 99 < 2^7 = 128$$

그래서 결과는 7비트면 충분하다. 하드웨어에서 $\times 10$ 은 시프트와 덧셈으로 만들 수 있다. $10T = 8T + 2T = (T \ll 3) + (T \ll 1)$ 이다. 예를 들어 $T = 4$ 이면 $32 + 8 = 40$ 이다.

유효성. 4비트는 0~15를 담을 수 있지만 BCD 한 자리는 0~9만 허용한다.

$$\text{valid} = (T \leq 9) \wedge (O \leq 9)$$

입력 조합 256개 중 유효한 것은 $10 \times 10 = 100$ 개, 무효는 156개다. 무효 입력일 때도 `bin` 은 $10T + O$ 를 계산하지만 7비트만 남으므로 128로 나눈 나머지가 나온다. 예를 들어 $T = O = 15$ 이면 $165 \bmod 128 = 37$ 이다. 이 값은 의미가 없으므로 `valid` 와 함께 읽어야 한다.

③ 포트 · 보드 매핑

신호	최상위 포트	보드 장치	FPGA 핀
ten[3:0]	sw[7:4]	DIP SW1~4	Y1 · W3 · U2 · T1
one[3:0]	sw[3:0]	DIP SW5~8	W4 · W1 · V4 · U4
valid	led[7]	LED1	L4
bin[6:0]	led[6:0]	LED2~8	M4 · M2 · N7 · M7 · M3 · M1 · N5

④ RTL

```
`timescale 1ns/1ps
// 출처: Glaysia/ece2_474 v1.1.0 docs/FPGA_additional.md (2. BCD를 이진수로 변환)
// 십의 자리와 일의 자리를 각각 0~9로 입력합니다.
module bcd_to_binary(
    input wire [3:0] ten, one,
    output wire [6:0] binary,
    output wire valid
);
    assign binary = ten * 7'd10 + one;
    assign valid = (ten <= 9) && (one <= 9);
endmodule
```

```
`timescale 1ns/1ps
// 출처: Glaysia/ece2_474 v1.1.0 docs/FPGA_additional.md
module bcd_conv(
    input wire [3:0] one, ten,
    output wire [6:0] bin,
    output wire valid
);
    bcd_to_binary u_convert(ten, one, bin, valid);
endmodule
```

```
`timescale 1ns/1ps
// 보드 연결: ten = SW1~4 (sw[7:4]), one = SW5~8 (sw[3:0])
//          valid = LED1 (led[7]), bin = LED2~8 (led[6:0])
module lab4_bcd_conv_top(input wire [7:0] sw, output wire
    bcd_conv u_bcd(.one(sw[3:0]), .ten(sw[7:4]), .bin(led[6:0]), .valid(led[7]));
endmodule
```

수업 코드의 `bcd_conv` 는 포트 순서가 `(one, ten, ...)` 이고 내부의 `bcd_to_binary` 는 `(ten, one, ...)` 이다. 순서가 달라 헷갈리기 쉬워서, 최상위 모듈에서는 이름으로 연결 (`.ten(...)`)했다.

⑤ 시뮬레이션

테스트벤치는 ten과 one을 각각 0~15로 바꿔 256개 조합을 넣는다. 매번 valid 가 맞는지, 유효한 입력이면 bin 이 $10T + O$ 인지 검사한다.

시뮬레이터	출력	판정
Vivado XSim	PASS BCD: 100 valid and 156 invalid inputs	통과
Icarus Verilog 12.0	PASS BCD: 100 valid and 156 invalid inputs	통과
보고서 생성 시 VCD 재검사	valid=1 구간 100개, 그중 bin 일치 100개 · 마지막 입력(15, 15)의 bin = 37 (=165 mod 128)	통과

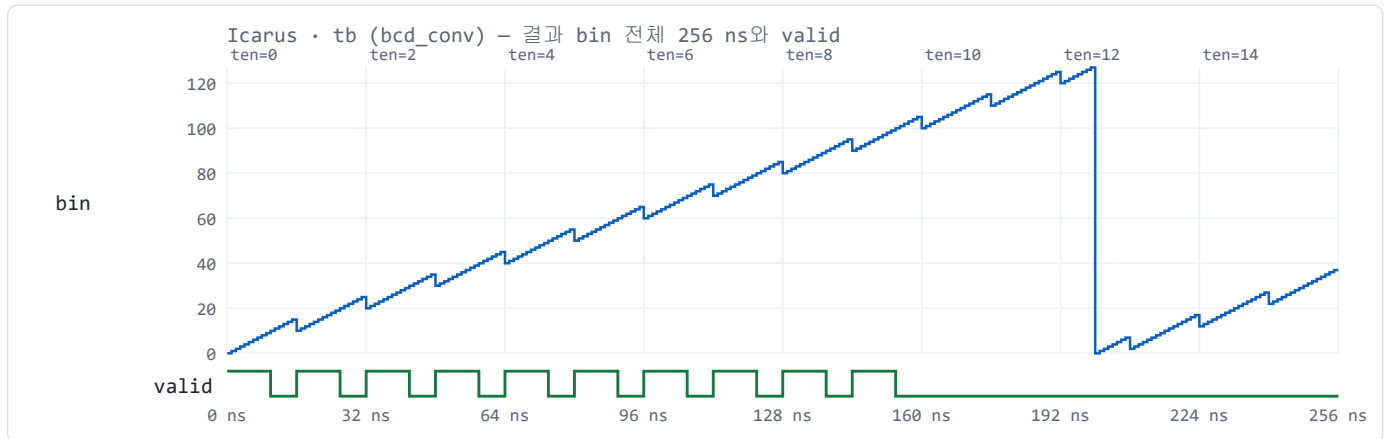


그림 2-1. 전체 파형. 각 16 ns 구간 앞쪽 10 ns(one=0~9)에서만 valid가 1이고, 160 ns 이후(ten≥10)는 valid가 계속 0이다. 160 ns 이후 bin이 아래로 꺾이는 것은 7비트를 넘어 128로 나눈 나머지가 되기 때문이다.

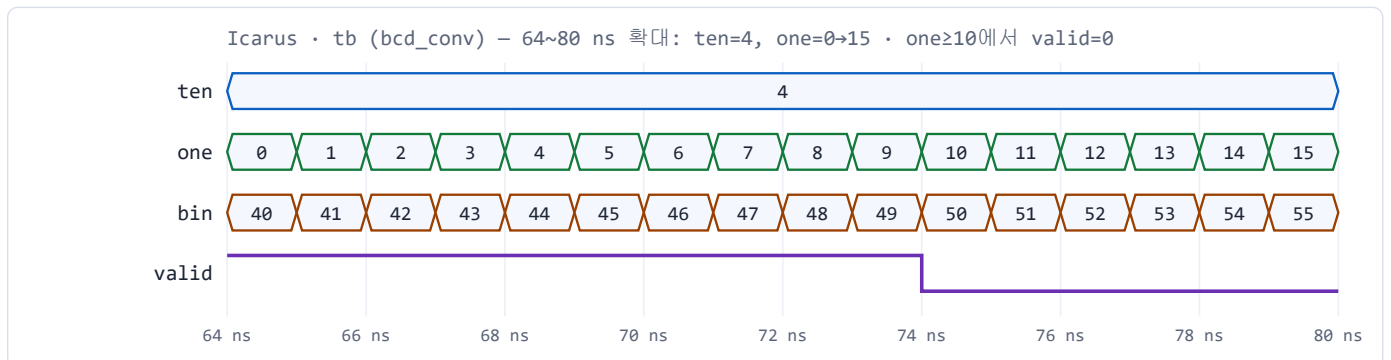


그림 2-2. ten=4 구간. one=0~9에서 bin=40~49, valid=1. one이 10이 되는 순간 valid가 0으로 내려간다.

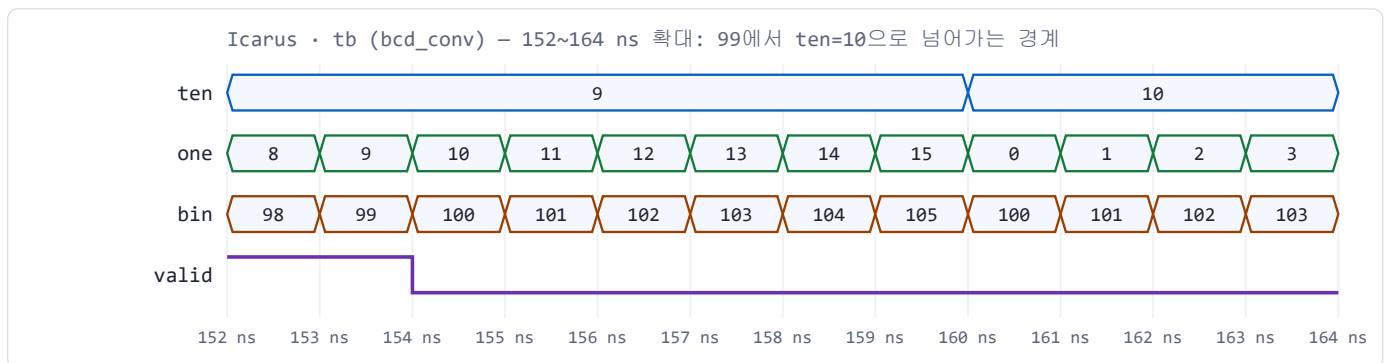


그림 2-3. 유효 범위의 끝. (9, 9)에서 bin=99, valid=1이고, 다음 구간 (10, 0)에서 valid=0이 된다.

⑥ 합성 · 구현 결과 (사전 빌드)

항목	값
합성 · 구현	synth_design Complete · write_bitstream Complete · bit_exists=1
Slice LUT	7 / 48,000 (0.01%)
Slice Register (FF)	0 — 조합회로
Bonded IOB	16 / 338
타이밍	WNS 0.000 / WHS 0.000 — 클록 없음

곱셈기(LUT 16)보다 작은 이유: $\times 10$ 은 상수 곱이라 "아무 수 $\times$ 아무 수"보다 회로가 훨씬 단순하다. 또 bin의 최하위 비트는 **101**가 항상 짝수이므로 one의 최하위 비트와 같아서 논리가 필요 없다.

⑦ 보드 실습 계획

LED1(초록 표시)이 valid, LED2~8이 bin이다.

ten, one	SW1~4 SW5~8	valid	bin	bin (이진)	LED1~8
0, 7	0000 0111	1	7	0000111	
4, 2	0100 0010	1	42	0101010	
9, 9	1001 1001	1	99	1100011	
10, 0 (무효 입력)	1010 0000	0	100	1100100	
3, 12 (무효 입력)	0011 1100	0	42	0101010	

마지막 두 줄은 일부러 넣은 무효 입력이다. LED1이 꺼지는 것이 핵심이고, LED2~8의 값은 의미 없는 나머지 값이다.

3. 실험 3 · STM32_0 과제1 — 버튼으로 LED 간격 변경

① 목적

NUCLEO-G474RE의 사용자 버튼(PC13)을 누를 때마다 내장 LED(PA5)의 상태 변경 간격을 $1\text{ s} \rightarrow 2\text{ s} \rightarrow 4\text{ s} \rightarrow 8\text{ s} \rightarrow 1\text{ s}$ 순서로 바꾼다. 버튼을 누르지 않아도 LED는 정해진 간격으로 계속 깜빡이고, 버튼을 누르고 있어도 간격은 한 번만 바뀌어야 한다.

② 이론 · 설계 계산

폴링 방식. FPGA는 회로가 항상 동시에 동작하지만, MCU는 CPU가 명령을 한 줄씩 실행한다. 메인 루프가 `polling_routine()` 을 계속 호출하고, 함수는 호출될 때마다 "버튼이 눌렀나?"와 "시간이 됐나?"를 차례로 확인하고 바로 돌아온다. 우체통을 수시로 열어 보는 것과 같다. 함수 안에서 기다리지 않기 때문에(`HAL_Delay` 없음) 버튼 확인과 LED 시간 확인이 서로를 막지 않는다.

시간 재기. `HAL_GetTick()` 은 1 ms마다 1씩 늘어나는 32비트 카운터 값을 돌려준다. HAL 기본 설정에서 이 증가 주기는 1 kHz이다 [2]. LED를 바꿀 때 시각 `last_ms` 를 저장해 두고, 지금 시각과의 차이가 간격 이상이면 다시 바꾼다.

$$\Delta = (\text{now} - \text{last}) \bmod 2^{32} \geq \text{interval} \Rightarrow \text{LED 반전}$$

부호 없는 32비트 뿔셈은 자동으로 2^{32} 로 나눈 나머지가 되므로, 카운터가 약 $2^{32}\text{ ms} \approx 49.7$ 일 뒤 0으로 넘어가도 차이는 올바르게 계산된다. 이 성질은 ⑤에서 모의 실행으로 확인했다.

간격과 깜빡임 주기. 간격은 켜짐→꺼짐 한 번의 시간이다. 켜짐과 꺼짐을 합친 깜빡임 한 주기는 간격의 두 배다.

$$T_{\text{blink}} = 2 t_{\text{interval}}, \quad t_{\text{interval}} \in \{1, 2, 4, 8\} \text{ s} \Rightarrow T_{\text{blink}} \in \{2, 4, 8, 16\} \text{ s}$$

간격 순환 규칙. 두 배로 늘리고, 8 s 다음에는 1 s로 돌아간다. 비교를 `== 8000U` 가 아니라 `>= 8000U` 로 써서, 값이 어떤 이유로 어긋나도 반드시 1 s로 돌아오게 했다.

한 번 누름 = 한 번 변경 (예지 검출). 버튼 값을 그대로 쓰면 누르고 있는 동안 계속 참이 되어 간격이 마구 바뀐다. 그래서 "지금은 눌림, 직전에는 안 눌림"인 순간만 인정한다. 매뉴얼 예제와 같이 이 보드의 버튼은 누르면 HIGH로 읽힌다.

$$\text{press} = (\text{current} = 1) \wedge (\text{previous} = 0) \wedge (\text{now} - \text{last_press} \geq 200 \text{ ms})$$

마지막 조건은 채터링 대책이다. 버튼 접점은 놀리는 순간 수 ms 동안 붙었다 떨어지기를 반복할 수 있다. 200 ms 안에 다시 검출된 누름은 무시한다.

③ 포트 · 보드 매핑

용도	MCU 핀	코드의 이름 (main.h)	설정
사용자 버튼 B1	PC13	<code>BUTTON_ece2_GPIO_Port / _Pin</code>	입력, 누르면 HIGH
내장 LED LD2	PA5	<code>LED_ece2_GPIO_Port / _Pin</code>	출력, SET=켜짐

MCU에는 XDC가 없다. 핀 기능은 수업 프로젝트의 CubeMX 설정(`ece2_474.ioc`)에서 생성된 초기화 코드가 정한다.

④ 핵심 코드

매뉴얼 제공 예제 1(1초 점멸)과 예제 2(버튼 예지 검출)를 한 함수에 합치고, 고정값 `1000U` 를 변수 `interval_ms` 로 바꿨다. ①은 이 변수를 바꾸고, ②는 이 변수를 읽는다.

```
void polling_routine(void)
{
    static uint32_t last_ms = 0;           // 마지막으로 LED를 바꾼 시각
    static uint32_t interval_ms = 1000U;  // 지금 간격 (처음은 1초)
    static GPIO_PinState previous = GPIO_PIN_RESET; // 직전 버튼 상태
    static uint32_t last_press_ms = 0;     // 마지막으로 버튼을 인정한 시각

    uint32_t now_ms = HAL_GetTick();       // 지금 시각 (ms)
    GPIO_PinState current = HAL_GPIO_ReadPin(BUTTON_ece2_GPIO_Port, BUTTON_ece2_Pin);

    /* ① 버튼 확인: 방금 눌린 순간이면 간격을 다음 값으로 */
    if (current == GPIO_PIN_SET && previous == GPIO_PIN_RESET
        && now_ms - last_press_ms >= 200U)
    {
```

```

    last_press_ms = now_ms;
    if (interval_ms >= 8000U)
    {
        interval_ms = 1000U;          // 8초 다음은 1초로 돌아감
    }
    else
    {
        interval_ms = interval_ms * 2U; // 1→2→4→8초
    }
}
previous = current;

/* @ 시계 확인: 간격만큼 지났으면 LED 반전 */
if (now_ms - last_ms >= interval_ms)
{
    last_ms = now_ms;
    HAL_GPIO_TogglePin(LED_ece2_GPIO_Port, LED_ece2_Pin);
}
}
}

```

⑤ 시뮬레이션 (호스트 모의 실행)

보드 없이 타이밍을 확인하려고, 위 `polling_routine()` 을 저장소 파일에서 그대로 잘라 PC의 gcc로 컴파일했다. `HAL_GetTick` · `HAL_GPIO_ReadPin` · `HAL_GPIO_TogglePin` 만 가짜 함수로 바꿨다. 가짜 시계는 1 ms씩 가고, 1 ms마다 함수를 3번 호출해 메인 루프가 빠르게 도는 상황을 흉내 냈다. 버튼 시나리오의 다음과 같다.

- 10.05 s에 눌러 1.5 s 동안 유지 → 간격이 한 번만 바뀌어야 함
- 30.05 s에 짧게 누름
- 60.05 s에 40 ms 누르고 떼 뒤, 3 ms · 7 ms 후 채터링처럼 접점이 두 번 더 붙음 → 한 번만 인정되어야 함
- 100.05 s에 누름 → 8 s 다음은 1 s

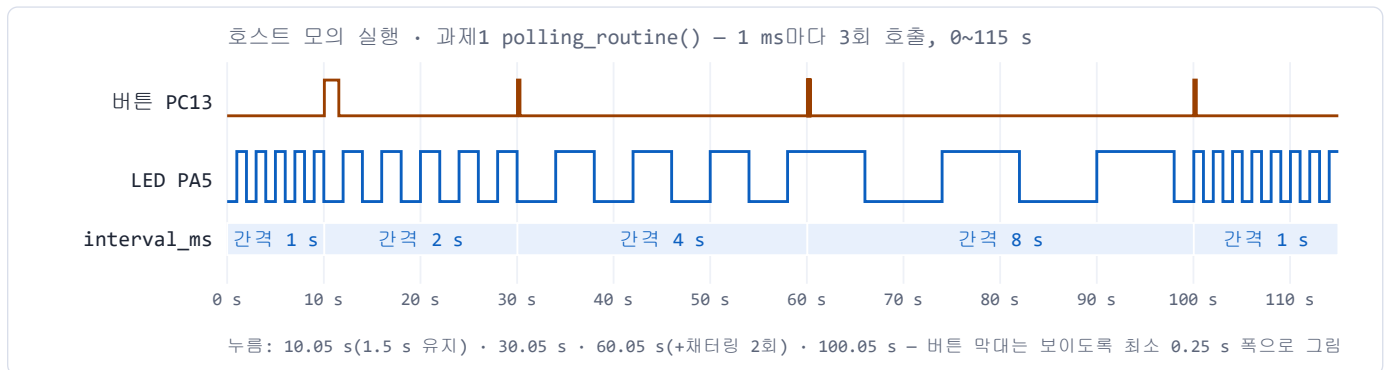


그림 3-1. 모의 실행 결과. LED 상태 변경 47회. 간격이 버튼을 누를 때마다 1→2→4→8→1 s로 바뀐다.

검사	기대	모의 실행 결과	판정
LED 변경 간격의 연속	1 s → 2 s → 4 s → 8 s → 1 s	1 s × 10 → 2 s × 10 → 4 s × 7 → 8 s × 5 → 2.05 s × 1 → 1 s × 14	일치
1.5 s 누르고 있기	한 번만 변경	1 s 다음 곧바로 2 s (4 s로 건너뛰지 않음)	일치
채터링 (60.05 s)	한 번만 변경	4 s 다음 8 s (1 s로 건너뛰지 않음)	일치
32비트 카운터 넘침	넘침 전후 같은 동작	시작 시각을 $2^{32} - 20001$ ms로 옮겨 다시 실행: 첫 변경 1회를 빼면 같은 간격 열 1 s × 10 → 2 s × 10 → 4 s × 7 → 8 s × 5 → 2.05 s × 1 → 1 s × 14	일치

예상과 다른 점 하나 (설계상 정상) — 표의 2.05 s × 1 은 100.05 s에 간격을 8 s에서 1 s로 줄이는 순간 생긴다. 마지막 변경(98 s) 뒤 이미 2.05 s가 지나 있었으므로 1 s 조건을 곧바로 만족해 LED가 누르는 즉시 한 번 바뀐다. 반대로 간격을 늘릴 때(예: 10.05 s)는 마지막 변경 시각부터 새 간격을 센다. 과제 조건(누를 때마다 간격 변경, 계속 깜빡임)은 모두 만족하므로 그대로 둔다. 보드에서 8 s → 1 s로 바꿀 때 LED가 바로 한 번 바뀌는 것이 정상임을 미리 알아 둔다.

⑥ 빌드 결과

항목	값
도구	VS Code + STM32Cube 확장 3.11.0 · CMake 4.4.0+st.1 · Ninja · gnu-tools-for-stm32 · preset Debug
결과	ece2_474.elf 생성, 종료 코드 0
FLASH	25,824 B / 512 KB (4.93%)
RAM	2,656 B / 128 KB (2.03%)
HAL 드라이버	STM32G4xx HAL v1.2.7 (프로젝트 포함본) [2]

⑦ 보드 실습 계획

1. JP5 점퍼가 5V_STLK 인지 확인하고 ST-LINK USB를 연결한다.
2. 매뉴얼 제공 예제 1(1초 점멸)과 예제 2(버튼으로 토글)를 각각 먼저 실행한다.
3. 과제1 코드로 교체 → 디버깅 종료 → **F5로 다시 다운로드** → main에서 멈추면 F5 한 번 더.
4. 스톱워치로 LED가 바뀌는 간격을 잰다. 버튼을 한 번씩 누르며 1 → 2 → 4 → 8 → 1 s를 확인한다.
5. 버튼을 2초 이상 누르고 있어도 간격이 한 단계만 바뀌는지 확인한다.

4. 실험 4 · STM32_0 과제2 — 외부 LED로 동작시키기

① 목적

과제1과 똑같은 동작을 보드 밖에 꽂은 LED로 보여 준다. 프로그램이 핀 이름만 바꾸면 다른 장치를 같은 방식으로 다룰 수 있다는 것을 확인한다.

② 이론 · 설계 계산

배선. 매뉴얼대로 PB0 → 저항 → LED 긴 다리(+), LED 짧은 다리(-) → GND로 연결한다. PB0이 HIGH(3.3 V)가 되면 전류가 흘러 LED가 켜진다. 저항은 전류를 제한하는 수도꼭지 역할이다.

$$I = \frac{V_{OH} - V_F}{R} \approx \frac{3.3 - 2.0}{R} \Rightarrow R = 330 \Omega : \approx 3.9 \text{ mA}, \quad R = 1 \text{ k}\Omega : \approx 1.3 \text{ mA}$$

$V_F \approx 2.0$ V는 빨간 LED를 가정한 값이다. 실제 LED 종류에 따라 다르며, 어느 경우든 수 mA 수준이라 밝기만 달라진다. 매뉴얼 권장 범위(330 Ω ~1 k Ω) 안에서 고른다.

코드 변경. 과제1 코드에서 LED를 바꾸는 한 줄의 포트 · 핀 이름만 PA5에서 PB0으로 바꾼다. HAL_GPIO_TogglePin 은 출력 데이터 레지스터(ODR)의 현재 값을 읽어 반대로 설정하는 함수라 [2], 어느 GPIO 핀에든 같은 방식으로 동작한다. PB0은 수업 프로젝트에서 이미 출력(초기 LOW)으로 설정되어 있다.

③ 포트 · 보드 매핑

용도	MCU 핀	코드의 이름	연결
사용자 버튼 B1	PC13	BUTTON_ece2	보드 내장
외부 LED	PB0	GPIO_OUT1_ece2_GPIO_Port / _Pin	PB0 → 330 Ω ~1 k Ω → LED(+) / LED(-) → GND

NUCLEO 보드에는 Arduino 표기(A0~A5, D0~D15)가 인쇄되어 있다. PB0의 헤더 위치는 실험 당일 보드 회로도(MB1367)로 확인한 뒤 꽂는다.

④ 핵심 코드 (과제1과의 차이)

```
- HAL_GPIO_TogglePin(LED_ece2_GPIO_Port, LED_ece2_Pin);
+ HAL_GPIO_TogglePin(GPIO_OUT1_ece2_GPIO_Port, GPIO_OUT1_ece2_Pin); /* 과제2: PB0 외부 LED */
```

나머지 줄은 과제1과 같다. 전체 파일은 저장소의 LAB4/stm32/task2/ece2_474_routine.c 에 있다.

⑤ 시뮬레이션 (호스트 모의 실행)

과제1과 같은 시나리오로 실행했다. 가짜 HAL_GPIO_TogglePin 이 어느 포트를 바꿨는지도 기록하게 해서, 47번의 변경이 모두 GPIOB(PB0)에서 일어나고 GPIOA(PA5)는 한 번도 바뀌지 않음을 확인했다.

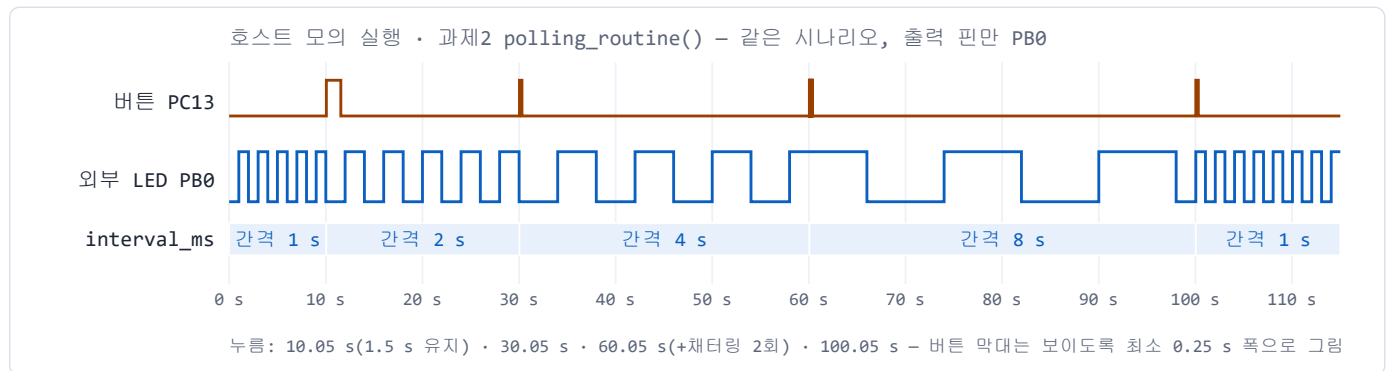


그림 4-1. 과제2 모의 실행. 간격 열은 1 s×10 → 2 s×10 → 4 s×7 → 8 s×5 → 2.05 s×1 → 1 s×14로 과제1과 같다.

⑥ 빌드 결과

종료 코드 0. FLASH 25,824 B (4.93%) · RAM 2,656 B (2.03%)로 과제1과 같다. 바뀐 것은 함수에 넘기는 포트 · 핀 상수뿐이기 때문이다.

⑦ 보드 실습 계획

1. 과제1을 보드에서 확인한 뒤 진행한다(매뉴얼 요구: 두 과제를 각각 확인).
2. 보드 전원을 끈 상태에서 PB0 → 저항 → LED → GND를 배선한다. LED 방향(긴 다리가 +)을 확인한다.
3. 과제2 코드로 교체 → F5 다운로드. 외부 LED가 1 s 간격으로 깜빡이는지 본다. 내장 LED는 바뀌지 않아야 한다.
4. 버튼으로 1 → 2 → 4 → 8 → 1 s 변경까지 확인한다.

참고문헌

1. [1] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed. Waltham, MA, USA: Morgan Kaufmann, 2013. ISBN 978-0-12-394424-5. — Ch. 1 수 체계(이진수 자리값), Ch. 5 Digital Building Blocks(곱셈기의 부분곱).
2. [2] STMicroelectronics, *stm32g4xx-hal-driver*, GitHub, <https://github.com/STMicroelectronics/stm32g4xx-hal-driver>, 태그 v1.2.7 (스타 53, 확인일 2026-10-07). — `stm32g4xx_hal.c` 의 `uwTickFreq` (기본 1 kHz) · `HAL_GetTick`, `stm32g4xx_hal_gpio.c` 의 `HAL_GPIO_TogglePin`.

수업 자료(인용 출처 수에 넣지 않음): Glaysia/ece2_474 v1.1.0의 README · docs/STM32_0.md · docs/FPGA_additional.md, 「전자전기컴퓨터설계실험 II STM32 실습 안내」 (2026-10-06), 실습 교재 「Verilog HDL을 이용한 디지털 논리 회로 설계」.